TEST TECHNIQUE

Développeur Full Stack

Projet : Communauté Le Revenu | Next.js · Strapi v5 · PostgreSQL

Contexte

Le Revenu est un média spécialisé dans l'information financière, les placements et l'investissement. Dans le cadre de l'évolution de notre plateforme numérique, nous souhaitons développer un espace communautaire permettant à nos lecteurs d'échanger, de poser des questions et de partager leurs connaissances.

Vous trouverez en pièce jointe une maquette HTML interactive représentant notre vision de cette communauté. Ce prototype, réalisé en HTML, CSS et JavaScript natif, comporte plusieurs écrans et interactions simulées mais n'est pas une application prête pour la production.

Votre mission : transformer une partie du prototype en une application web structurée et maintenable, avec Next.js pour le frontend et Strapi v5 pour le backend.

1. Stack technique imposée

Frontend : Next.js (App Router), React, TypeScript, Tailwind CSS.

Backend : Strapi v5, PostgreSQL, TypeScript, API REST exposée par Strapi.

L'utilisation des fonctionnalités natives de Strapi est attendue : Content Types, relations, controllers, services, policies et permissions. La logique métier et la persistance doivent passer par Strapi. Next.js ne doit pas accéder directement à PostgreSQL ni implémenter un backend métier parallèle.

Vous pouvez ajouter des bibliothèques complémentaires en justifiant leur utilisation.

2. Fonctionnalités à développer

A. Accueil de la communauté

- Afficher des discussions issues de l'API Strapi.
- Présenter le titre, l'auteur, la catégorie, le nombre de réponses et la date de publication.
- Filtrer les discussions par catégorie et trier par date de publication.
- Permettre la navigation vers le détail d'une discussion.

Les discussions ne doivent pas être codées en dur dans les composants React.

B. Détail d'une discussion

- Créer une URL propre à chaque discussion.
- Afficher le titre, le contenu, l'auteur, la date, la catégorie et les réponses associées.
- Permettre la publication d'une réponse enregistrée dans PostgreSQL via Strapi.

C. Création d'une discussion

- Formulaire avec titre de 10 caractères minimum, contenu obligatoire, catégorie et type (question ou discussion).
- Afficher les erreurs de validation et rediriger vers la nouvelle discussion après création.
- Appliquer les validations métier côté Strapi, indépendamment des contrôles du frontend.

D. Utilisateurs et permissions

L'authentification complète n'est pas obligatoire. Vous pouvez utiliser Users & Permissions de Strapi avec des utilisateurs de démonstration.

- Associer chaque discussion et chaque réponse à un utilisateur.
- Interdire la publication aux utilisateurs non authentifiés.
- Empêcher un utilisateur de publier au nom d'un autre.
- Configurer les permissions côté Strapi et non uniquement dans le frontend.

3. Architecture backend Strapi

Concevez au minimum les entités et relations suivantes :

- User : privilégier le modèle utilisateur du plugin Users & Permissions.
- Category : name, slug, description.
- Discussion : title, content, type, author (User), category (Category), replies (Reply).
- Reply : content, author (User), discussion (Discussion).

Vous pouvez ajouter des champs, des contraintes et des index pertinents. Les endpoints natifs sont autorisés lorsqu'ils suffisent ; pour les règles métier spécifiques, utilisez des controllers, services ou policies Strapi.

Nous attendons une récupération optimisée des relations, une pagination et un tri côté backend, une validation des entrées, des autorisations et un traitement cohérent des erreurs HTTP. Le Document Service API de Strapi v5 est recommandé pour les opérations métier personnalisées.

4. Architecture frontend Next.js

- Composants React réutilisables et responsabilités bien séparées.
- Appels à Strapi centralisés et typés.
- Choix cohérent entre Server Components et Client Components.
- Gestion des états de chargement, erreur et absence de données.
- Interface responsive et respect global de la maquette HTML.

Vous devrez pouvoir expliquer les choix liés au rendu serveur, à la récupération des données et à leur actualisation.

5. Bonus facultatifs

- Recherche de discussions.
- Votes « Utile » et meilleure réponse.
- Pagination des discussions et profil utilisateur.
- Tests end-to-end avec Playwright.

- Docker Compose pour Next.js, Strapi et PostgreSQL.

La qualité du périmètre principal prime sur le nombre de bonus réalisés.

6. Tests automatisés

Fournissez quelques tests pertinents, notamment sur la validation d'une discussion, le refus d'une publication non authentifiée, la protection contre l'usurpation d'auteur et l'affichage des discussions. Le choix des outils est libre.

7. Livrables attendus

- Dépôt Git contenant les projets Next.js et Strapi v5.
- Configuration PostgreSQL et schémas des Content Types.
- Mécanisme d'initialisation des données de démonstration.
- Fichier .env.example sans secrets.
- README.md expliquant installation, lancement et choix d'architecture.
- Tests automatisés réalisés.

Le projet doit pouvoir être lancé localement en suivant le README.

8. Modalités

La reproduction intégrale de la maquette n'est pas demandée :
privilégiez un périmètre cohérent, correctement construit et maintenable.

Les outils d'assistance au développement, y compris l'intelligence artificielle, sont autorisés. Vous devrez néanmoins comprendre et justifier le code remis.

Une revue technique de 30 minutes permettra d'échanger sur votre implémentation et les possibilités d'évolution.

Bon développement !

L'équipe technique — Le Revenu